Assignment 03 - Smart Tank Monitoring System

Cattolico Giuseppe, Muller Arthur

2 aprile 2026

Indice

1 Introduzione e Obiettivo **3**

2 Architettura Hardware **4**
2.1 Schema Elettrico e Breadboard 4
2.2 Componenti e Pin Mapping – TMS (ESP32) 4
2.3 Componenti e Pin Mapping – WCS (Arduino UNO) 4

3 Architettura Software **5**
3.1 Struttura Generale del Sistema 5

4 TMS – Tank Monitoring Subsystem (ESP32) **5**
4.1 Task Concorrenti 5
4.2 FSM di Connettività (ConnectionTask) 5
4.3 Astrazione Hardware 6

5 WCS – Water Channel Subsystem (Arduino) **6**
5.1 Task Concorrenti 6
5.2 FSM del Water Channel (FSMController) 6
5.3 Comunicazione Seriale Asincrona – MsgService 7

6 CUS – Control Unit Subsystem (Java/Vert.x) **7**
6.1 Framework e Architettura 7
6.2 Protocolli e Classi di Comunicazione 7
6.3 API REST (DataService Verticle) 7
6.4 Logica di Controllo Automatico 8
6.5 Design Pattern Adottati 8

7 DBS – Dashboard Subsystem (Web) **8**
7.1 Visualizzazione 8
7.2 Controllo 9

8 Pattern Comuni ai Sottosistemi Embedded **9**

9 Difficoltà Riscontrate e Conclusioni **9**
9.1 Difficoltà Riscontrate 9
9.2 Conclusioni 10

1 Introduzione e Obiettivo

Il progetto *Smart Tank Monitoring System* ha l'obiettivo di realizzare un sistema IoT distribuito per il monitoraggio del livello dell'acqua piovana in un serbatoio e per la gestione automatizzata del canale di drenaggio. Il sistema è suddiviso in quattro sottosistemi che cooperano attraverso protocolli eterogenei: MQTT, comunicazione seriale e HTTP.

- **TMS** (Tank Monitoring Subsystem) – ESP32: acquisisce il livello del serbatoio e lo trasmette via MQTT.
- **WCS** (Water Channel Subsystem) – Arduino UNO: controlla la valvola del canale tramite servomotore, reagendo ai comandi del backend.
- **CUS** (Control Unit Subsystem) – Backend Java: orchestra l'intero sistema, applicando la politica di controllo e servendo le API HTTP.
- **DBS** (Dashboard Subsystem) – Frontend Web: fornisce un'interfaccia per operatori remoti per monitorare e interagire col sistema.

Il sistema opera in due modalità: `AUTOMATIC`, in cui il backend decide autonomamente l'apertura della valvola in base al livello dell'acqua, e `MANUAL`, in cui un operatore controlla direttamente la valvola tramite potenziometro (WCS) o dashboard (DBS). La modalità di avvio è `AUTOMATIC`.

2 Architettura Hardware

2.1 Schema Elettrico e Breadboard

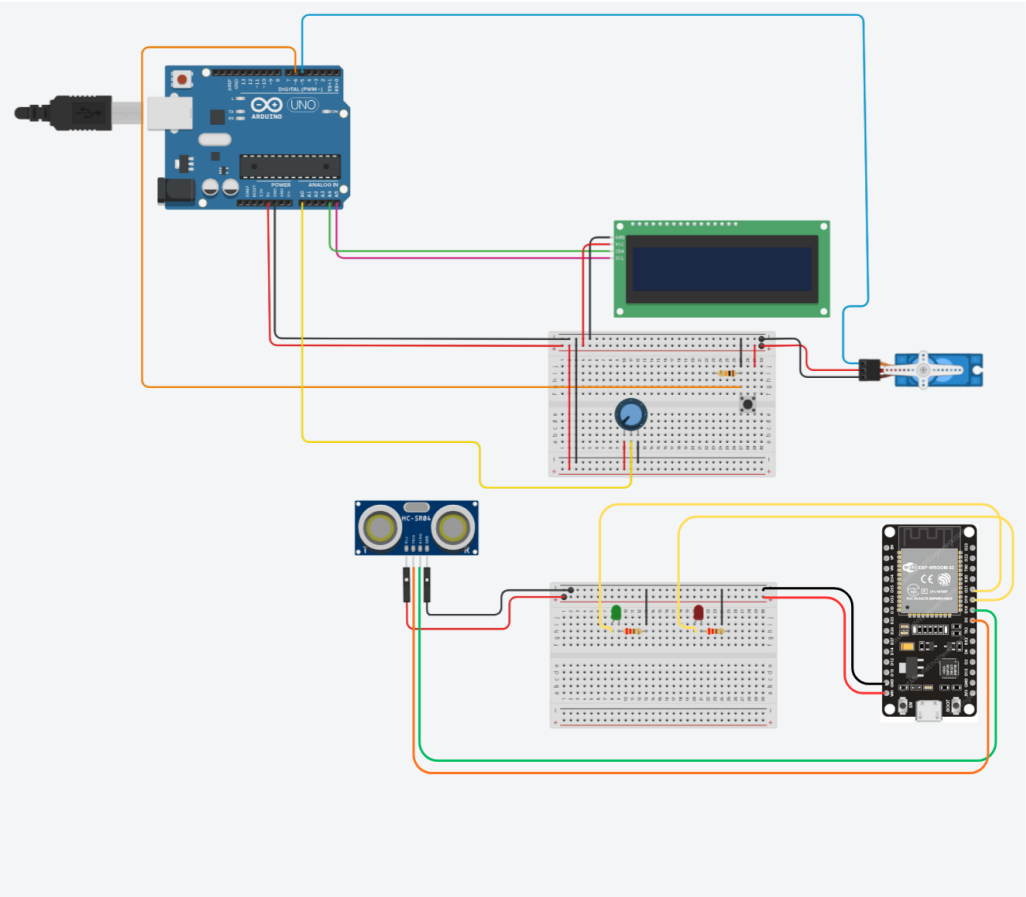

Figura 1: Rappresentazione della breadboard e schema circuitale (TMS e WCS).

2.2 Componenti e Pin Mapping – TMS (ESP32)

- **Sonar HC-SR04** – `Trig: Pin 5, Echo: Pin 18`: Misura il livello dell'acqua nel serbatoio tramite impulsi ultrasonici. Il valore di distanza viene periodicamente campionato e inviato al CUS via MQTT.
- **LED Verde** – `Pin 17`: Acceso quando la connessione Wi-Fi e MQTT è operativa e l'invio dei dati avviene correttamente.
- **LED Rosso** – `Pin 19`: Acceso in caso di problemi di rete (Wi-Fi o MQTT non disponibili), segnalando all'operatore locale uno stato di errore.

2.3 Componenti e Pin Mapping – WCS (Arduino UNO)

- **Servomotore (valvola)** – `Pin 10`: Controlla l'apertura del canale idraulico. L'intervallo operativo va da 0 (valvola chiusa, 0% apertura) a 90 (valvola completamente

aperta, 100% apertura). Utilizza la libreria `ServoTimer2` per evitare conflitti con `TimerOne`.

- **Potenziometro** – `Pin A0`**:** Permette il controllo manuale dell'apertura della valvola in modalità `MANUAL`. Il valore analogico letto viene normalizzato nell'intervallo $[0.0, 1.0]$, con mapping invertito ($0\% \rightarrow 90$, $100\% \rightarrow 0$) per rispettare la meccanica del sistema.
- **Pulsante Tactile** – `Pin 8`**:** Commuta tra le modalità `AUTOMATIC` e `MANUAL`. Alla prima pressione si entra in `MANUAL`; alla seconda si torna in `AUTOMATIC`.
- **Display LCD I2C (16x2)** – `Indirizzo 0x27`**:** Mostra il livello attuale di apertura della valvola in percentuale e la modalità operativa corrente (`AUTOMATIC`, `MANUAL` o `UNCONNECTED`).

3 Architettura Software

3.1 Struttura Generale del Sistema

Tutti i sottosistemi embedded (TMS ed WCS) condividono lo stesso pattern architetturale: uno scheduler cooperativo basato su interrupt hardware (libreria `TimerOne`) con periodo base di **50ms**, task periodici definiti come sottoclassi di `Task`, e un oggetto `Context` come repository condiviso dello stato tra i task. Il backend Java è basato su **Vert.x** con architettura event-driven e comunicazione asincrona non bloccante.

4 TMS – Tank Monitoring Subsystem (ESP32)

4.1 Task Concorrenti

Il TMS è strutturato in tre task cooperativi gestiti dallo scheduler:

- **SensorTask (periodo: 200ms):** Campiona il sonar HC-SR04 applicando una compensazione della temperatura per aumentare la precisione della misura. Il valore letto viene salvato nel `Context` condiviso, rendendolo disponibile al `PublishTask`.
- **PublishTask (periodo: 2000ms):** Legge il livello del serbatoio dal `Context` e lo pubblica sul broker MQTT. Aggiorna il LED verde o rosso in base all'esito della pubblicazione.
- **ConnectionTask (periodo: 1000ms):** Implementa una FSM a tre stati per la gestione della connettività. Si occupa di rilevare e recuperare automaticamente la perdita della connessione Wi-Fi o MQTT.

4.2 FSM di Connettività (ConnectionTask)

La macchina a stati gestisce tre stati distinti:

- `CHECKING`: stato nominale. Verifica periodicamente che la connessione Wi-Fi e MQTT sia attiva. Se viene rilevata una disconnessione, transita nel rispettivo stato di riconnessione.

- `RECONNECTING_WIFI`: tenta il ripristino della connessione Wi-Fi. Al successo, transita verso `RECONNECTING_MQTT`.
- `RECONNECTING_MQTT`: tenta la riconnessione al broker MQTT. Al successo, ritorna in `CHECKING`.

Le azioni di *entry* in ogni stato sono gestite tramite il pattern `justEntered` + `checkAndSetJustEnte` che garantisce l'esecuzione unica delle operazioni di inizializzazione alla prima iterazione del task nello stato.

4.3 Astrazione Hardware

I device fisici sono nascosti dietro interfacce astratte (`ProximitySensor`, `Light`), con implementazioni concrete (`Sonar`, `Led`). La classe `TankMonitoringPlatform` funge da *facade* per l'inizializzazione e l'accesso centralizzato all'hardware.

5 WCS – Water Channel Subsystem (Arduino)

5.1 Task Concorrenti

Il WCS è strutturato in tre task cooperativi:

- **FSMController (periodo: 75ms):** Task principale che implementa la FSM dell'intero sottosistema. Gestisce le transizioni tra le modalità operative, aggiorna la valvola e il display LCD, e reagisce agli input del pulsante e ai comandi seriali.
- **PotReader (periodo: 200ms):** Campiona il valore del potenziometro e lo normalizza nell'intervallo $[0.0, 1.0]$, salvandolo nel `Context`. Attivo solo quando la FSM si trova in modalità `MANUAL`.
- **SerialTask (periodo: 50ms):** Gestisce la comunicazione bidirezionale con il CUS tramite porta seriale a 115200 baud. Riceve comandi di modalità e percentuale di apertura valvola, e trasmette lo stato corrente del WCS.

5.2 FSM del Water Channel (FSMController)

La macchina a stati gestisce tre stati operativi:

- `AUTOMATIC`: la valvola è controllata interamente dal CUS tramite comandi seriali. Il display mostra la percentuale di apertura corrente e la dicitura `AUTOMATIC`. La pressione del pulsante provoca la transizione in `MANUAL`.
- `MANUAL`: l'operatore locale controlla direttamente la valvola tramite il potenziometro. Il display mostra la percentuale corrente e la dicitura `MANUAL`. La pressione del pulsante riporta alla modalità `AUTOMATIC`. In questo stato, il WCS notifica al CUS la modalità attiva per sincronizzare il sistema distribuito.
- `UNCONNECTED`: il CUS non è raggiungibile (timeout seriale). Il display mostra `UNCONNECTED`. Il sistema mantiene l'ultima posizione nota della valvola fino al ripristino della connessione.

Soglie di controllo automatico (CUS → WCS):

- Livello $> L_1$ (20%) per più di T_1 secondi → valvola al **50%**.
- Livello $> L_2$ (30%) → valvola al **100%** immediatamente.
- Livello rientrato sotto la soglia → valvola richiusa.

5.3 Comunicazione Seriale Asincrona – MsgService

La comunicazione con il CUS avviene in modo non bloccante tramite la classe `MsgService`. I caratteri in arrivo vengono accumulati in un buffer; quando viene rilevato il terminatore `\n`, la stringa è impacchettata in un oggetto `Msg` e un flag `msgAvailable` viene alzato. Il `SerialTask` verifica il flag ad ogni ciclo e processa il messaggio senza mai bloccarsi in attesa. Il protocollo adottato è basato su stringhe del tipo `MODE,VALVE:XX`, dove `XX` rappresenta la percentuale di apertura target.

6 CUS – Control Unit Subsystem (Java/Vert.x)

6.1 Framework e Architettura

Il backend è sviluppato con **Vert.x 4.2.6**, un framework event-driven e non bloccante per la JVM. Vert.x permette di gestire simultaneamente le tre interfacce di comunicazione del CUS (MQTT, seriale, HTTP) senza ricorrere a thread multipli bloccanti, grazie al modello basato su *Verticle* e *Event Bus*.

6.2 Protocolli e Classi di Comunicazione

- **MQTTProtocol**: utilizza il client *Eclipse Paho* in modalità callback. Sottoscrive il topic pubblicato dal TMS e aggiorna il timestamp dell'ultimo dato ricevuto per rilevare situazioni di `UNCONNECTED` (timeout T_2).
- **SerialProtocol**: utilizza la libreria `jssc` con un `SerialPortEventListener` per la ricezione asincrona dei dati dal WCS. Una `BlockingQueue` funge da buffer di decoupling tra il thread seriale e il loop Vert.x, garantendo thread-safety.
- **HTTPProtocol**: esposto tramite `Vert.x Web`, serve le API REST consumate dalla dashboard. Utilizza `Vert.x WebClient` per eventuali chiamate HTTP interne in modalità non bloccante con futures/callback.

6.3 API REST (DataService Verticle)

Il `DataService` espone i seguenti endpoint HTTP:

- `GET /api/data` – restituisce le ultime N misurazioni del livello del serbatoio (buffer circolare in memoria, `LinkedList` con massimo 10 elementi).
- `GET /api/status` – restituisce lo stato corrente del sistema: `AUTOMATIC`, `MANUAL` o `UNCONNECTED`.
- `POST /api/valve` – permette alla dashboard di impostare la percentuale di apertura della valvola in modalità `MANUAL`.

6.4 Logica di Controllo Automatico

Il `CUS` implementa la policy di controllo del livello secondo le seguenti regole:

1. Se il livello supera L_1 per più di T_1 secondi, il CUS invia al WCS il comando di aprire la valvola al 50%. La valvola viene richiusa quando il livello scende sotto L_1.
2. Se il livello supera L_2 (indipendentemente dal tempo), la valvola viene aperta al 100% immediatamente. Viene richiusa quando il livello scende sotto L_2.
3. Se non vengono ricevuti dati dal TMS per più di T_2 secondi, il sistema transita nello stato `UNCONNECTED`, sospendendo le operazioni automatiche. Il ripristino avviene non appena i dati tornano disponibili.

6.5 Design Pattern Adottati

- **Callback/Listener**: per operazioni asincrone su MQTT e seriale.
- **Observer**: per la propagazione degli eventi di cambio modalità (`ModeChangeCallback`) e cambio valvola (`ValveChangeCallback`).
- **Message Queue** (`BlockingQueue`): per il disaccoppiamento tra il thread seriale di `jssc` e il thread principale Vert.x.
- **Protocol Adapter**: ciascun protocollo (MQTT, seriale, HTTP) è incapsulato in una classe dedicata che adatta le chiamate native all'interfaccia del CUS.
- **Façade** (`CUSService`): centralizza l'orchestrazione dei tre protocolli, nascondendo la complessità interna.

7 DBS – Dashboard Subsystem (Web)

La dashboard è implementata come **applicazione web statica** (HTML + CSS + JavaScript puro), senza dipendenze da framework pesanti. Effettua polling verso le API del CUS ogni secondo.

7.1 Visualizzazione

- **Grafico livello serbatoio**: realizzato con la libreria `Chart.js`, mostra le ultime N misurazioni ricevute dal CUS in forma di grafico a linea aggiornato in tempo reale.
- **Percentuale apertura valvola**: indicatore numerico e/o barra di progressione che riflette lo stato attuale della valvola.
- **Stato del sistema**: badge testuale colorato che mostra `AUTOMATIC`, `MANUAL`, `UNCONNECTED` o `NOT AVAILABLE` (quest'ultimo quando il CUS non è raggiungibile).

7.2 Controllo

- **Pulsante modalità**: commuta tra `AUTOMATIC` e `MANUAL` inviando una richiesta `POST` all'endpoint `/api/status` del CUS.
- **Slider valvola**: visibile e attivo solo in modalità `MANUAL`. Permette di impostare la percentuale di apertura desiderata, inviando la richiesta `POST /api/valve` al backend.

8 Pattern Comuni ai Sottosistemi Embedded

I sottosistemi TMS (ESP32) e WCS (Arduino) condividono un'identica architettura interna, che garantisce coerenza, riutilizzabilità e facilità di manutenzione:

1. **Cooperative Task Scheduler**: scheduler basato su interrupt `TimerOne` con periodo base di 50ms. Ogni task dichiara il proprio periodo e viene invocato esclusivamente quando il suo intervallo è scaduto. Nessun task è bloccante.
2. **FSM Enum-Based**: le macchine a stati sono implementate tramite `enum` C++ e blocchi `switch/case`, senza overhead di ereditarietà. Le azioni di entry vengono eseguite una sola volta grazie al pattern `justEntered`.
3. **Context / Shared State**: un unico oggetto `Context` per sottosistema funge da repository condiviso. I task non comunicano mai direttamente tra loro: scrivono e leggono esclusivamente tramite il `Context`, eliminando ogni accoppiamento diretto.
4. **Façade Pattern** (`Platform`): le classi `TankMonitoringPlatform` e `WaterChannelPlatform` centralizzano l'inizializzazione e l'accesso all'hardware, nascondendo i dettagli di configurazione dei pin.
5. **Interface Segregation**: ogni device fisico è rappresentato da un'interfaccia astratta (`Light`, `ProximitySensor`, `Button`) con una o più implementazioni concrete. Questo permette di sostituire o simulare i componenti hardware senza modificare la logica dei task.
6. **Message Passing Asincrono**: la comunicazione con il backend avviene tramite `MsgService`, che garantisce ricezione non bloccante via buffer e flag di disponibilità.

9 Difficoltà Riscontrate e Conclusioni

9.1 Difficoltà Riscontrate

- **Conflitti tra librerie Timer su Arduino:** L'utilizzo simultaneo di `TimerOne` (per lo scheduler) e della libreria standard `Servo` ha generato conflitti sui timer hardware dell'ATmega328P. Il problema è stato risolto adottando `ServoTimer2`, che utilizza `Timer2` lasciando `Timer1` libero per lo scheduler.
- **Sincronizzazione thread seriale e Vert.x:** La libreria `jssc` esegue i callback di ricezione seriale in un thread separato rispetto al loop Vert.x. Questo ha reso necessario l'uso di una `BlockingQueue` per il passaggio sicuro dei messaggi tra i due contesti di esecuzione, evitando race condition.

- **Gestione dello stato UNCONNECTED distribuito:** Coordinare la transizione in `UNCONNECTED` in modo coerente tra CUS (timeout MQTT), WCS (timeout seriale) e DBS (CUS non raggiungibile) ha richiesto un'attenta definizione dei timeout e dei meccanismi di ripristino per ciascun sottosistema.

- **Latenza del polling dashboard:** Il polling HTTP ogni secondo introduce una latenza percettibile nella visualizzazione degli aggiornamenti. Per un sistema di produzione si valuterebbe l'adozione di WebSocket o Server-Sent Events per aggiornamenti in push.

9.2 Conclusioni

Lo sviluppo dello *Smart Tank Monitoring System* ha dimostrato come un'architettura distribuita e modulare – basata su task cooperativi, FSM leggere e design pattern consolidati – permetta di realizzare un sistema IoT reattivo e manutenibile anche con risorse hardware limitate.

La separazione netta delle responsabilità tra i quattro sottosistemi ha semplificato lo sviluppo parallelo e il debugging: ciascun componente poteva essere testato indipendentemente prima dell'integrazione finale. I pattern comuni condivisi tra TMS e WCS hanno ridotto significativamente la duplicazione del codice e reso immediata la comprensione di entrambi i firmware.

Il sistema finale rispetta i requisiti funzionali: monitoraggio continuo del livello, controllo automatico e manuale della valvola, rilevamento di perdita di connettività e dashboard web in tempo reale.